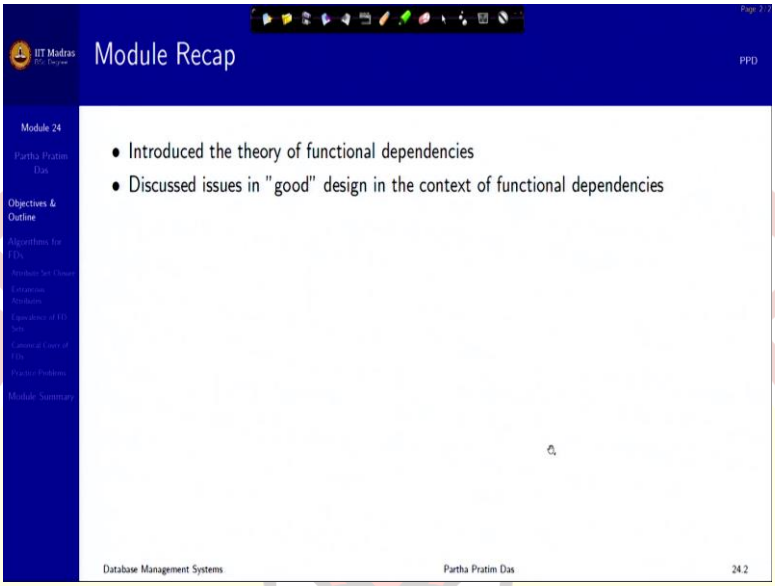
IIT Madras

ONLINE DEGREE

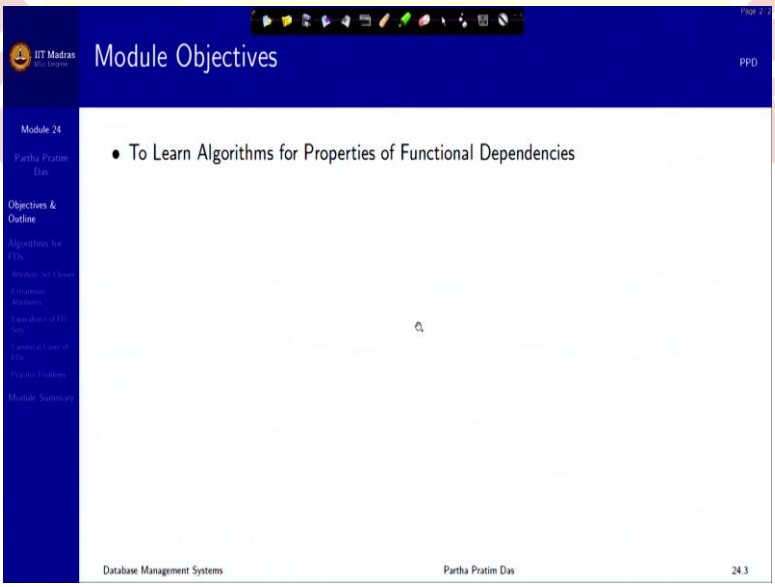
Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/4

(Refer Slide Time: 0:31)



Welcome to Module 24 of Database Management Systems course, in IIT Madras, online BSc program. We have been discussing about the design of relational databases and we introduced the notion of functional dependencies, which is key in building a mathematical framework for this design theory. We have talked about some basic information, definitions on functional dependencies and some notions of attribute closure and stuff like that.

(Refer Slide Time: 1:01)



Module Outline

Module 24
Partha Pratim Das
Objectives & Outline
Algorithms for FDs
Attribute Set Closure
Extraneous Attributes
Equivalence of FD Sets
Canonical Cover of FDs
Practice Problems
Module Summary

- Algorithms for Functional Dependencies

Algorithms for Functional Dependencies

Module 24
Partha Pratim Das
Objectives & Outline
Algorithms for FDs
Attribute Set Closure
Extraneous Attributes
Equivalence of FD Sets
Canonical Cover of FDs
Practice Problems
Module Summary

Algorithms for Functional Dependencies

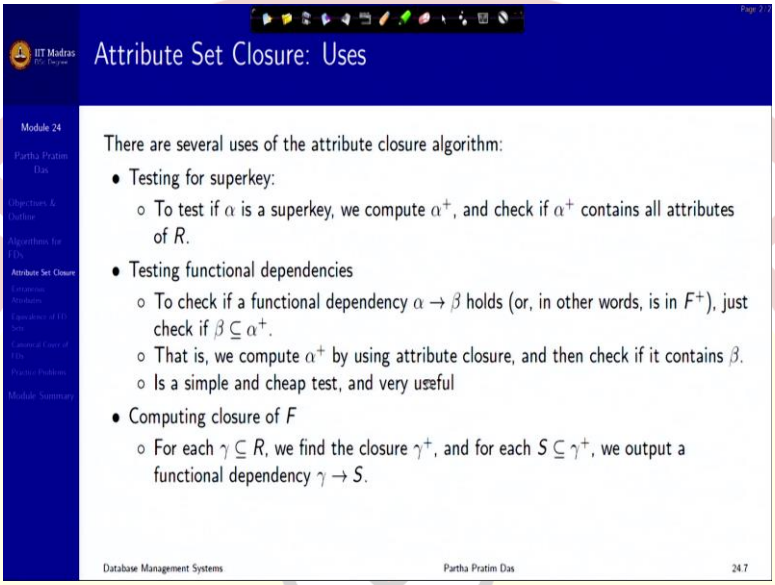
Attribute Set Closure

Module 24
Partha Pratim Das
Objectives & Outline
Algorithms for FDs
Attribute Set Closure
Extraneous Attributes
Equivalence of FD Sets
Canonical Cover of FDs
Practice Problems
Module Summary

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  a) $result = AG$
  b) $result = ABCG$ $(A \rightarrow C$ and $A \rightarrow B)$
  c) $result = ABCGH$ $(CG \rightarrow H$ and $CG \subseteq AGBC)$
  d) $result = ABCGHI$ $(CG \rightarrow I$ and $CG \subseteq AGBCH)$
- Is AG a candidate key?
  a) Is AG a super key?
     i) Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$
  b) Is any subset of AG a superkey?
     i) Does $A \rightarrow R$? == Is $(A)^+ \supseteq R$
     ii) Does $G \rightarrow R$? == Is $(G)^+ \supseteq R$

In this module, we are going to learn different important algorithms that can compute properties of functional dependencies. So, the first is the closure of attribute set which we have already studied. We have seen this example before, where we can take any subset of attributes and by repeatedly applying the functional dependencies on to it, till no change can happen, we can compute the closure set of the set of attributes, any set of attributes.

(Refer Slide Time: 1:53)

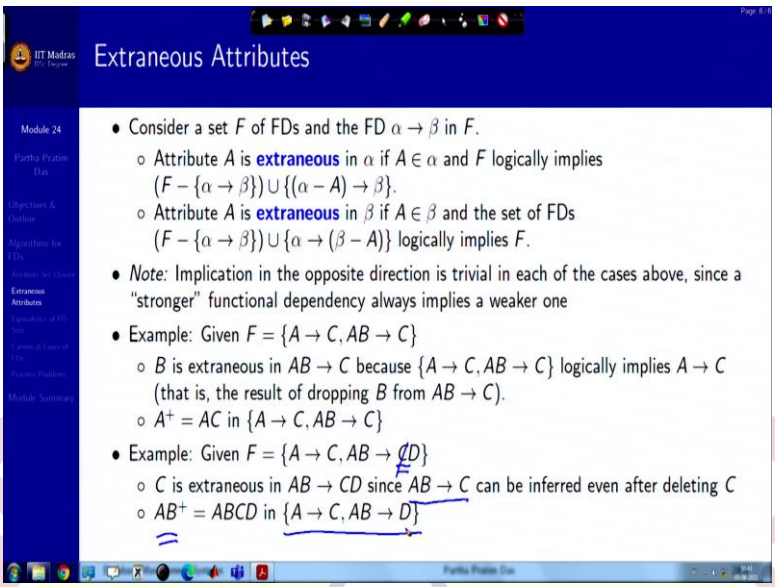


So, there we have also discussed that there are several uses for this closure attribute, closure set computation. And today actually, we will see, going forward we will see more and more of these use. First is, obviously testing for super key because it, a set of attributes can be a super key if its closure is the entire set of attributes.

We can also check for candidate key by taking subsets of a super key and checking if that too is a super key or not. We can check for functional dependencies that is to check whether alpha determines beta, all that we need to do is to compute the attribute closure for the set alpha which is α+ and check whether we beta is an subset of that, if it is, then the functional dependency holds, if it is not, then it does not hold.

And also in terms of computing the closure of the functional dependencies F, you remember we have introduced the closure of functional dependencies that is a set of functional dependencies F. We can compute F+ as the closure set, which is all possible functional dependencies that may be logically implied from the set F and we can also for taking different subsets of the relation set, we can find the respective closure and, in that process, using that we can check for different functional dependencies in the closure set and compute the closure set.

(Refer Slide Time: 3:40)



So, this is one use, now we will go into defining other notions and checking how to compute them. The first that we introduced is called the extraneous attribute. A functional dependency may be such that it has extra attributes and attributes which may not actually be required. So, it is kind of a sense to minimise a functional dependency and this extraneous functional dependency, extraneous attribute in a functional dependency could be either on the left hand side or it could be on the right hand side.

So, we will illustrate two examples, but just to look at the definition, suppose we have a functional dependency alpha determines beta in F, where alpha is a subset of the set of attributes, beta naturally is in another subset of the set of attributes. Then you say a particular attribute A is extraneous in this functional dependency, if obviously, if it belongs to the left hand side, that the first case.

That is A is an element of alpha, and if I remove it, if I remove it, still then the set of functional dependencies will not change. So, what is the factor of removing it, first is I removed the functional dependency from the set F, F – {α determines β}, for my new functional dependency, where A is not there on the left hand side, so, that is (α – A) determines β, we are checking if this is possible to be done and added to the set.

And now, if this set and the set F are equivalent, or if F logical implies this set, then we can say that A is extraneous and can be removed. In similar terms, we can find an attribute A in the right hand side beta obviously has to belong to beta and check, what is the effect of removing A in beta.

So, I removed the original functional dependencies, and we add a new functional dependency where alpha determines beta minus A, beta difference A, and see whether that can logically imply F. So, you can you can see the two sides. Because, what you are doing is in the first case, in the left, when it is in the left hand side, you are removing some attributes from the left hand side.

So, you are actually making the functional dependencies stronger, because you need less number of attributes to match, to declare that the right hand side beta matches. So, you are making it stronger. What you were doing in the extraneous on the second case, on the right hand side, you are actually removing an attribute from beta, which means that it is actually getting weakened, we have this is a weaker dependency, because earlier you are saying that alpha can functionally determine all attributes of beta.

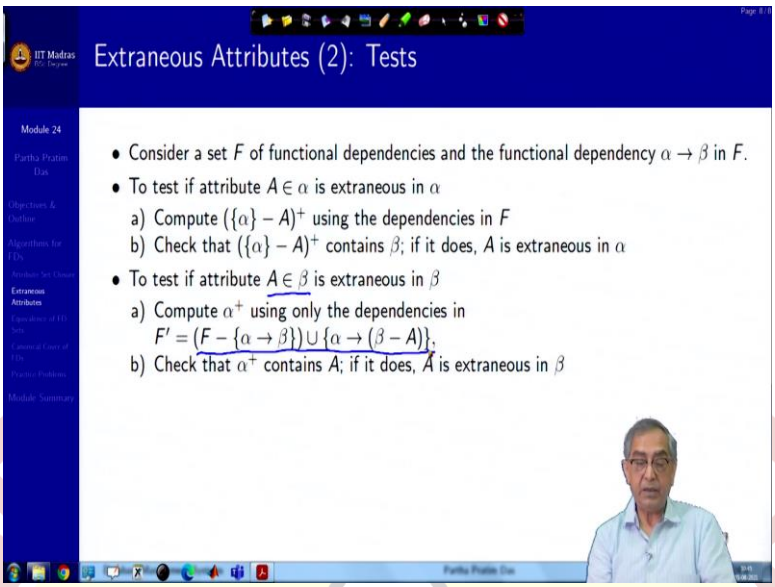
Now, you are saying that is not needed, if it just does beta minus A will that also be equivalent, so you have a weaker functional dependency. So, you can try to see that, in some cases, it will be possible to remove, in some cases it will not be possible to remove. So let us look at this, let us take a set of functional dependencies, A determine C, AB determine C.

Now, we are checking for left hand side, let us say is B extraneous in AB determine C. The question is if I take this set of functional dependencies F, then this implies A determines C that is, if I drop B, I am checking whether A determined C is implied. Now that is already there. So, what you can check by doing a AC?

So, basically, what we conclude is we can remove the dependent, the attribute B from the left hand side. This might look like a pathological example, but that is what often comes up. Now, let us say if we choose something on the right hand side, A determine C and AB determines C, CD. Now, we want to say that C is extraneous in this AB determine CD, because AB determine C can be inferred even after deleting C from the right hand side. Why? Because A determine C, if A determine C, then obviously AB determines C, because you are making the functional dependency weaker.

So, it can be inferred, since it can be inferred I can get AB determine C and I can, I will have AB determines D. So, by union rule, I will have AB determine CD. So, you can also do this by doing the attribute closure, which is the formal way you will do it and you will see that this is, this is what is possible and I can remove C from the right hand side.

(Refer Slide Time: 9:40)

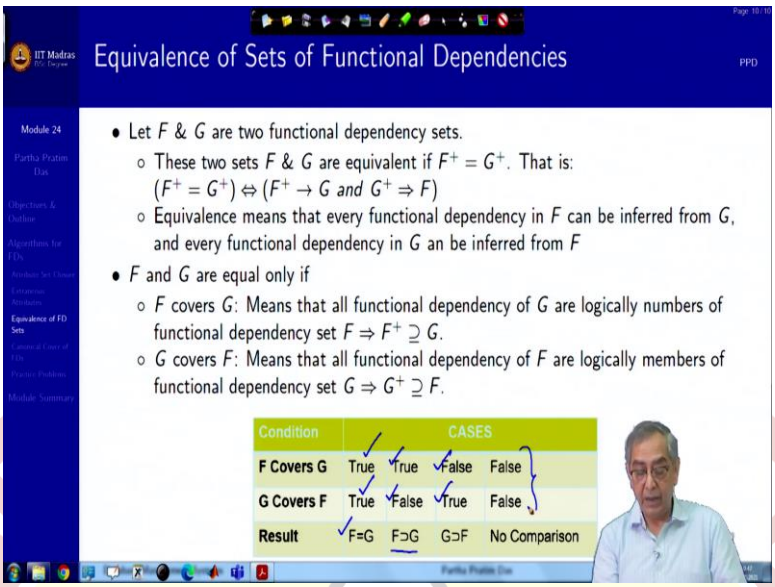


So, this is, so you can see that it is possible to add some cases, possible to remove some extraneous attributes on the left hand side or the right hand side and the tests are simple. So, to test if A is extraneous in alpha that is on the left hand side, all that you need to do is to take out A from alpha and compute the closure using all functional dependencies in F. Because you have taken out and then this is, this is what you will have. And you will check whether this closure of attributes contains beta, if it does A is extraneous, very straightforward.

For the other, it will have to be done a little differently, because you are now making having a weaker set. So, a weaker set whether it will imply the stronger set, so, you make the new set and call it F prime, let us say. So, with respect to F prime, now, you compute the closure of α, α+.

Earlier you were, in the first case you were competing with respect to F, because you had, you had made it, you had made it stronger. Now, you are computing it with respect to F prime, which is a new set and if that contains that F alpha plus contains say A, then you will say that A is extraneous in beta and can be removed. So, the tests are very simple using the closure of the attributes algorithm.

(Refer Slide Time: 11:22)



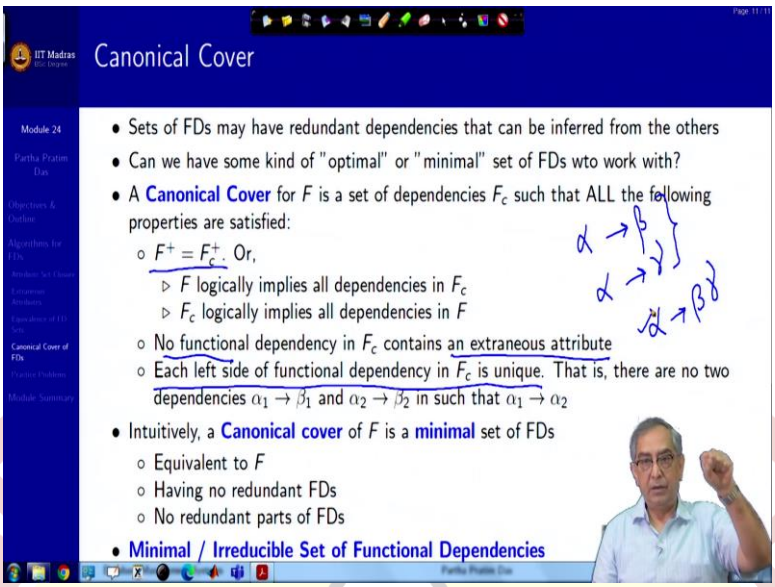
Naturally, the other notion which you have already understood is that, how do we set if two sets of functional dependencies are equivalent? You can say one is stronger than the other and the other is stronger than this one. So, that is the, that is a simple way. So, two, I am sorry, there are two sets F and G, we will say they are equivalent if their closure is same.

That if I take the closure of F and if I take the closure of G, then if they are identical, then obviously these are equivalent, because when I have a set F of functional dependencies, what actually works is a set F+ similarly for G. So, it there could be two sides to it, you have to prove by the cover that one is F+ implies G that is, if you compute F+, G will be a subset of that at the same time.

So, that says F covers G and the other side is G+ implies F, which means G+ will contain F, that is G covers F, and if both of them are true, then you say that they are equivalent. So, these are the possibilities if F covers G is true, and this is true, G covers F is true, then is, then these sets are equivalent. If F covers G is true, but G covers F is false, then G is weaker. G, so F is a superset of G, though, actually, this is not a set operation, because the sets are different.

But in the notion, the strength that F has G does not have that much strength. So, actually the subset relationship, we will be able to see if we actually compute the closure and we will see that F+ is a superset of G+ and the other side where G covers F. And if both of them are false that is none covers the other then naturally there is no comparison there are, they are independent sets of functional dependencies.

(Refer Slide Time: 13:47)



Now, we come to the notion of canonical cover. So, you can see that a set of functional dependencies is not something which is unique. I can remove some attributes on the left hand side that changes, I can remove some attributes from the right hand side. When I remove some attributes or even otherwise, it may be possible that there is some functional dependency which is not actually required to be there, but can be implied from others.

So, if we, if you think in that, you know in the, in the intuitive sense, then what we are saying is the closure set is a maximal, this is given this F, this is the whole world of functional dependencies it means. Now, we want to go to the other extreme and that is more required for efficiency of algorithms that, what is the minimal set? What are the functional dependencies that I must have, so that I retain the original set of constraints?

So, to define, so, that is what we define as canonical cover you may have come across this term in say switching theory if you have done, canonical is the standard cover. So, there are three basic requirements, one, is that obviously, this is obvious. So, F is a given set and FC is the canonical minimal set, which is expected to be smaller different from F.

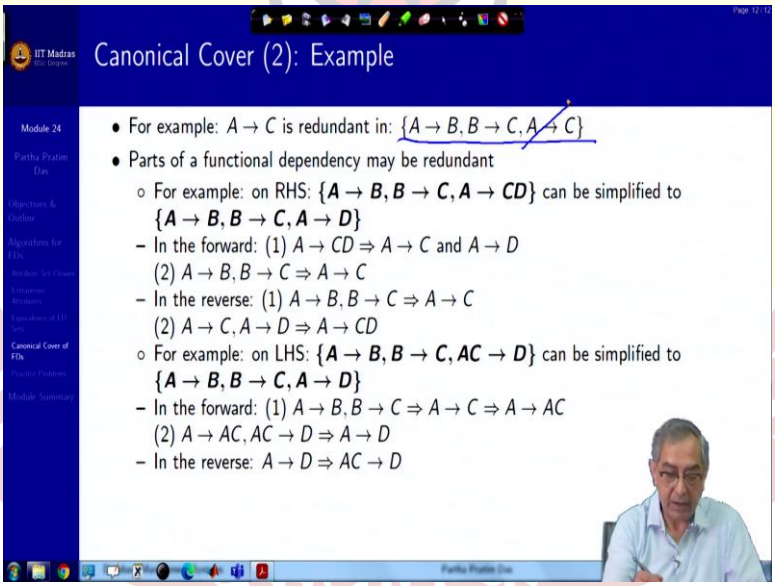
Now, naturally they have to be equivalent, if they are not equivalent, it does not work as a as a cover, so, that is, that goes without saying which means F must logically imply all dependencies in FC, FC must logically imply all dependencies in F we have just seen. No functional dependency should have an extraneous attribute, there should be no extra fat in the body, there should be every functional dependency should be as slim as possible both on the left hand side as well as on the right hand side.

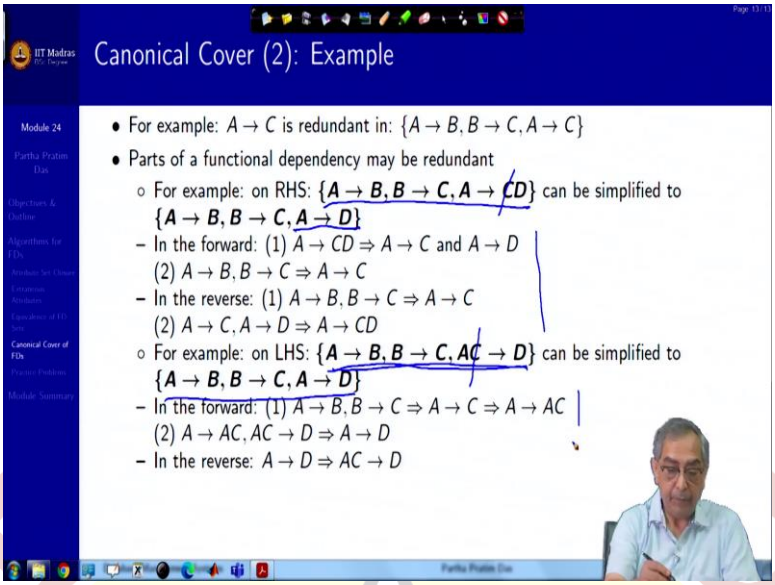
And each left hand side of a functional dependency is unique that is there would not be two different functional dependencies say alpha determining beta and alpha determining gamma in the set. If it exists, we can always use the union rule and write it like this. So, we will have this so, it is possible that if actually I have multiple functional dependencies with identical left hand side, I will use the union rule, make them together into one set and have less number of functional dependencies.

So, intuitively canonical cover is a minimal set of functional dependencies, it is equivalent to F it does not have any redundant functional dependency, it does not have any redundant part in a functional dependency in terms of extraneous attributes. Now, this, it is also called besides canonical cover, it is also called minimal set, it is also called irreducible set because it cannot be made smaller.

And please note that again we are using the term minimal, we are not saying minimum, because it is possible that you will have more than one canonical cover for a set of functional dependencies.

(Refer Slide Time: 17:31)





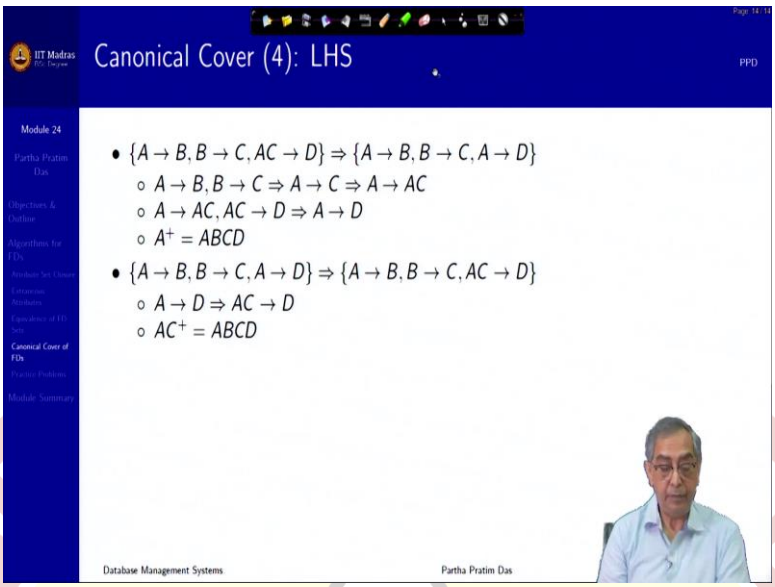
Example, so, let us say I am given with this set of functional dependencies, A determines B, B determines C, A determines C. Now, naturally, the functional dependency A determines C is redundant here, why is it redundant? Because it can be inferred from the A determines B and B determined C. So, if I if I remove this, it still remains the same, it still gives the same closure for the functional dependencies and therefore, it is a minimal set.

Now, for removing parts, let us say we consider this set A determines B, B determines C, A determines CD. Now, this can obviously you can see that, this can be removed, the C can be removed, because A determines B, B determines C means that A determines C. So, if A determines C it obviously, A determines C and A determines D after removal if I have them then I can always have, I will always have A determines CD.

So, this is now naturally you will have to show it both ways that this actually works on on both ways and conclude that D can be, C can be removed from the right hand side. To remove on the left hand side, let us consider this after this reduction has been done. Then on I am sorry, on a different example, A determines B, B determines C and AC determines D.

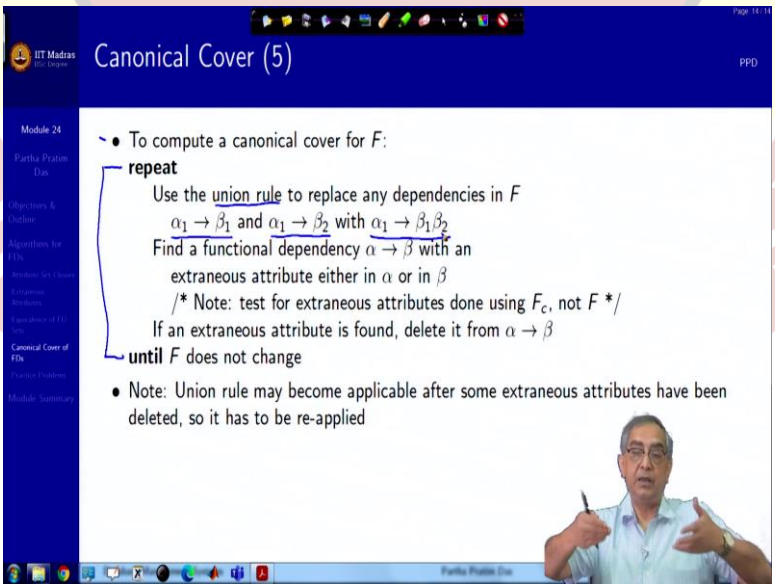
Now, you can easily observe that C can be removed from here, why? Because A determines B, B determines C transitively says A determines C. So, if A determines C, then having C on the left hand side is not required. Because if tuples match on A, they will obviously match on C. So, the C is redundant, you can also use Armstrong's axioms to deduce. So, this the canonical cover turns out to be A determines B, B determines C and A determines D. You can again prove it in both ways.

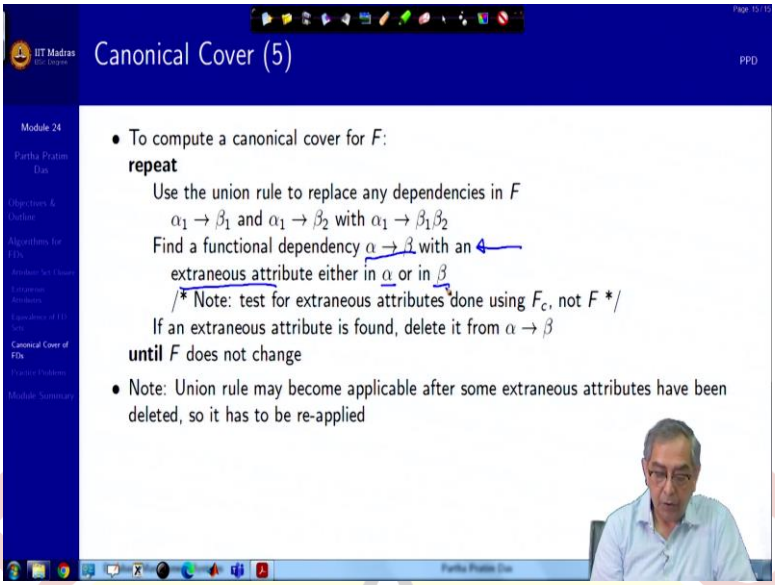
(Refer Slide Time: 19:57)

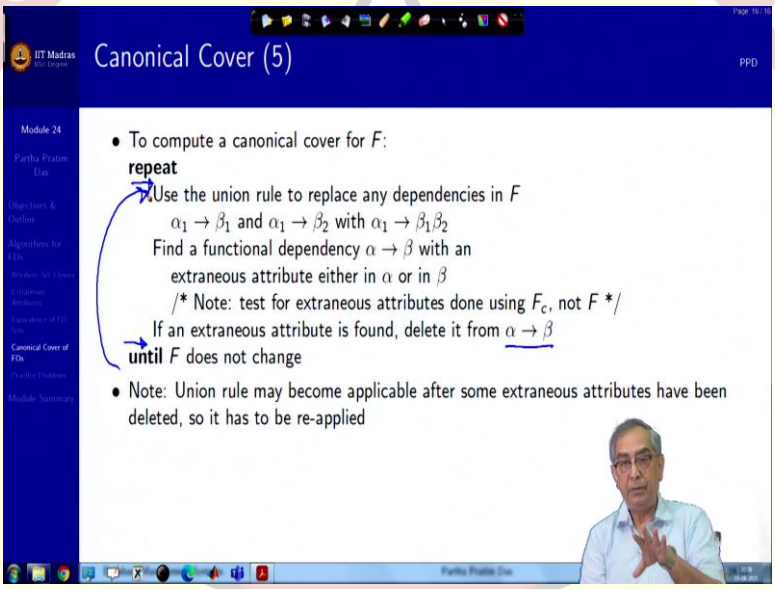


So, this is how the canonical cover works and it strongly helps in solving several problems here, I have shown the other way of doing the same solution instead of using Armstrong's axioms and doing a kind of ad hoc observation based logic, this is the structured algorithm where you do the same thing using the closure of attributes, and you can show that the minimality of the canonical cover reduction in these cases. Please try and practice it at home. So, this is for the other example. So, I am not going through the steps of this.

(Refer Slide Time: 20:36)







Now, let us look at if if this is the process, how do we structure it in terms of algorithm? Because certainly, we would like to automate each and every part of the design process, so that complex designs can be easily handled. So, this is the, this is the iteration that we will be doing. Now, the first thing we say is, at every stage, we tried to see if there are two or more functional dependencies with the identical left hand side.

So, we will use union rule, if alpha 1 determines beta 1, and alpha 1 determines beta 2, we will replace it by alpha 1 determines beta 1 beta 2, at every stage, because there may be new cases that are, that the new functional dependencies that arise, that has the same left hand side as a functional dependency already existing in the set, that is why it is to be done repeatedly.

So, this will ensure that at this point, the set of functional dependencies have unique left hand side. Now, you find a functional dependency alpha determines beta which has an extraneous

attribute either in alpha or in beta. Now, you certainly have to do this using not the original F, because you have changing it. So, you have to do it with the current Fc canonical cover that you have.

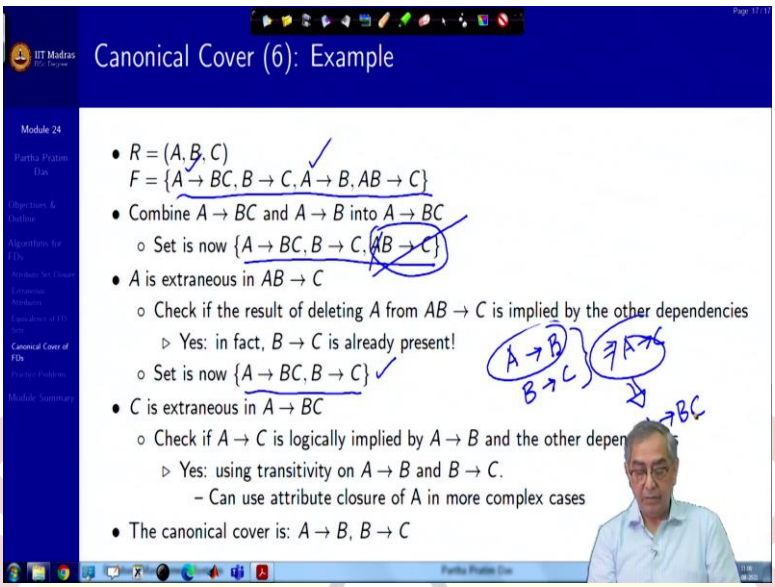
And you have already seen the algorithm to determine extraneous attribute, take every functional dependency and consider left hand side, check if there is an attribute which is extraneous, check the right hand side if there is any attribute which is extraneous, if it is there, you remove that and you get. So, if you find an extraneous attribute, then you delete it from alpha determines beta.

Now, through these two steps, the set may have changed. The, from whatever you started the iteration at this point, coming to this point, it has changed. If it has changed, then you go back and do this again. Again, check for identical left hand side, check for, do that change, get a new Fc with respect to that check again, if there are extraneous attributes on left and right. If it is there, remove, and in this process, if change has happened, then you keep repeating.

Now, obviously, this repetition, is possible, this repetition is terminable, because whenever you give an algorithm, we have to ensure that it terminates, this algorithm is terminable, for the simple reason that in every stage, you are either reducing one depend, one functional dependency, or you are reducing one attribute, at least. If you have not reduced any functional dependency and if you have not reduced any attribute, then obviously we will have an identical set.

So, in each iteration, the you will be reducing at least one from each and therefore the whole set keeps on shrinking. And since it is a finite set, it will at some point stop changing, so that you have got the minimal cover, the canonical cover. Note the order in which you do these operations might impact the actual result that you produce. So, it is possible that on different order of application you may have different canonical cover results.

(Refer Slide Time: 25:09)



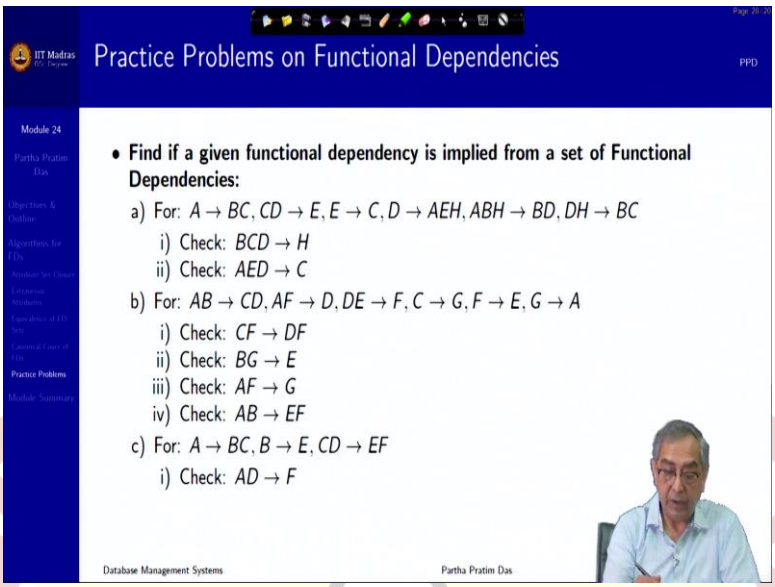
So, this is an example which I would encourage you to work through, this is worked according to the algorithm. So, this is the set we start with. So, what all we can, we can combine these two, because have identical left hand side. So, if we do that, then we have A determines BC and A determines B. So, that gets combined and we have a smaller set.

Now, A is extraneous in AB determine C, yow will come, come through the algorithmic check, but here by observation we can very easily say because B determine C, so, AB will obviously determine C. So, if you remove this then you have B determine C, which is already present. So, it actually once you remove this the entire functional dependency gets removed and this is your reduced set.

Now, what do you have? You have A determines BC and B determine C. Now, in A determines BC, we claim that C is extraneous, because, if I remove that I have A determines B, I already have B determines C. So, these two together will give me A determine C by transitivity and then these two together gives me A determines BC. The closure will, closure of attributes will make it easy to use this, but this is the basic process.

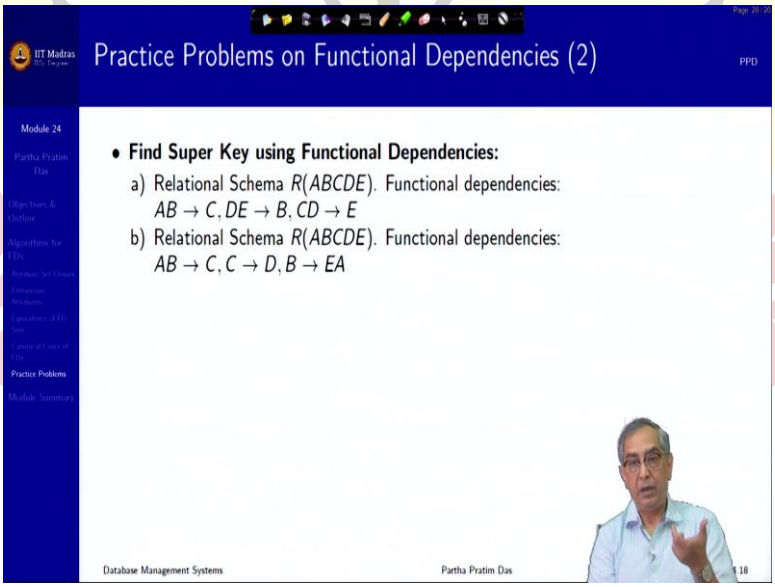
So, this can, from this C can also be removed. So, what I am left with, is this, it is interesting to see, you had quite a lot and now, we are just left with two functional dependencies each having only one element on left and right, two unique one, so, this is the minimal cover. So, that brings us to I mean, what I have given after this is a set of problems for you to practice based on the algorithms that we have done.

(Refer Slide Time: 27:20)



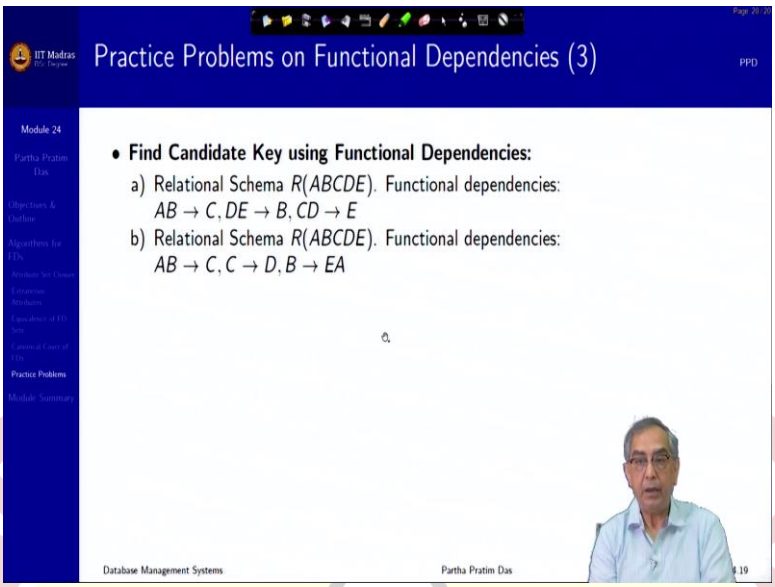
The first set of problems is to find a given functional, whether a given functional dependency is implied from a set of functional dependencies. So, you know, what needs to be done, you have to do a closure of the left hand side set of attributes and see whether the right hand side attribute is set is included in that. So, there are a couple of these, please practice so that you will start really understanding the algorithm.

(Refer Slide Time: 27:58)



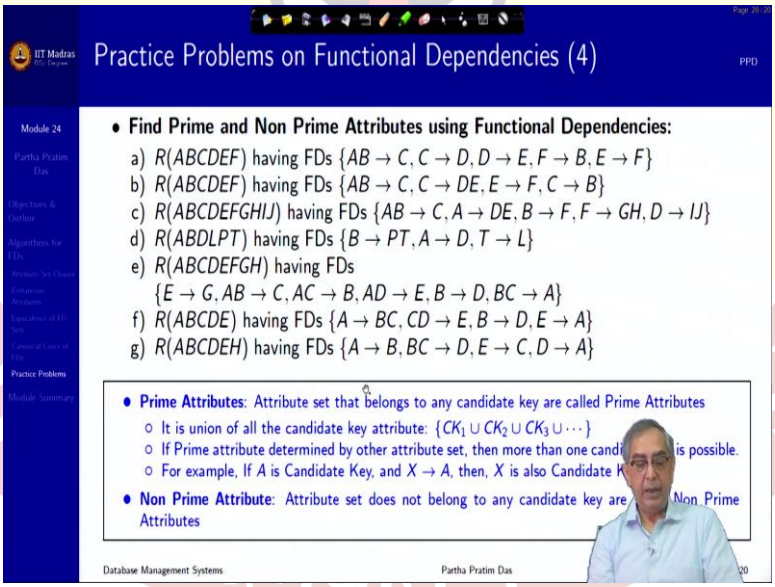
The second set of problems relate to finding super key. So, you have to, given a set of functional dependencies, you have to find a the sets which are super key which means the sets of attributes for which the closure is the entire set of attributes.

(Refer Slide Time: 28:21)



The third set of problems are similar to the second set, here you have to find the candidate keys that is super key, which is minimal. So, candidate key, so you have to show that it, this is a super key and no subset of it is a super key.

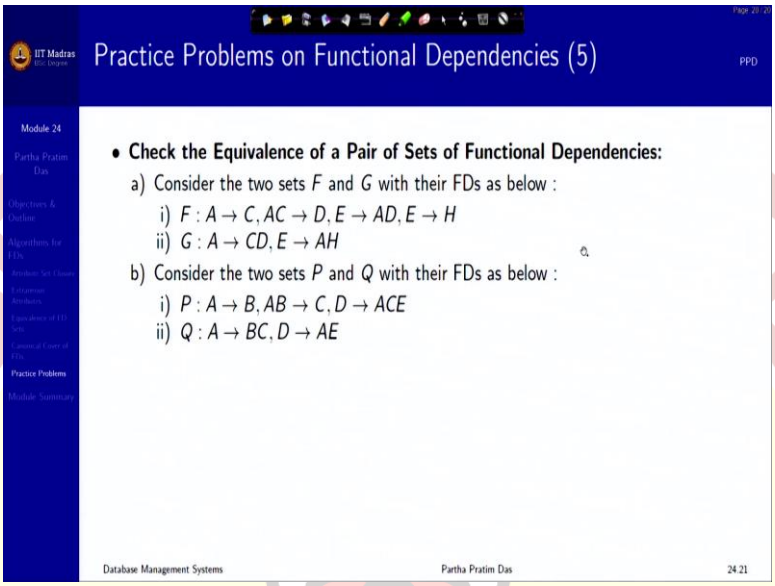
(Refer Slide Time: 28:40)



The fourth set is about prime and non-prime attributes. The prime attribute is, is attribute set that belongs to any candidate key, if some attribute belongs to some candidate key, then it is called a prime attribute. So, if you have, CK1, CK2, CK3 different candidate keys then you take the union of all of them, that set is a prime, set of prime attributes and non-prime attributes are those which does not belong to any candidate key.

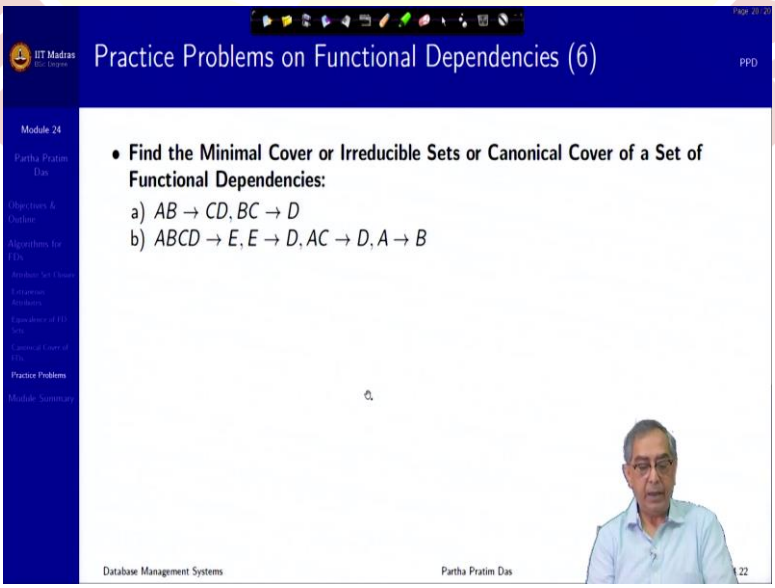
These are called non-prime attributes these notions are critical, because we are going to use them subsequently. And in this problem set you have to find the prime and non-prime attributes of the following different functional dependencies sets to get a good hold on the notion.

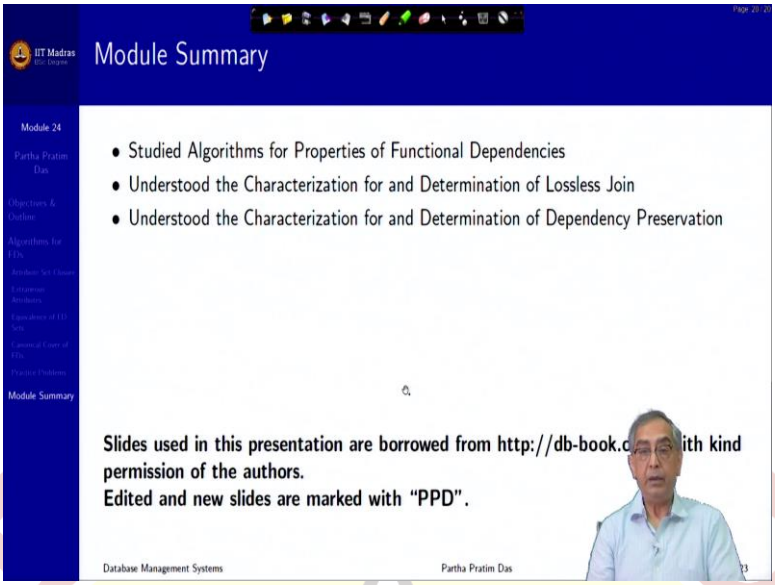
(Refer Slide Time: 29:28)



Then checking equivalence, we have already discussed how to check the equivalence and you will try to use the attribute closure algorithm to check the equivalence. That is, you have to show that for for F covering G you have to show that F can, say in a functional dependency in G say, A determine CD can be logically implied from F and so on. If we try to just do the whole cover and compute, it goes out of hand.

(Refer Slide Time: 30:03)





And then some problems for finding the minimal cover as we have just done. So, there is a list of problems. They are there, and you have to solve them at home and get comfortable. So, this brings us to the end of this module. Thank you for your attention, and we will continue this in the next module.